Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра обчислювальної техніки

Дискретна математика

Лабораторна робота №2

«Бінарні відношення та їх основні властивості, операції над відношеннями».

Виконав:
студент групи ІО-45
Унятицький А. Д.
Перевірив Пономаренко А.М.

Київ – 2024

Лабораторна робота №2

Тема: «Бінарні відношення та їх основні властивості, операції над відношеннями».

Мета: вивчити основні властивості бінарних відношень та оволодіти операціями над бінарними відношеннями.

Загальне завдання:

1. Написати в окремому модулі функцію для формування несуперечливих бінарних відношень.

2. Написати в окремому модулі функції виконання логічних операцій над бінарними відношеннями.

3. Пояснити правило формування несуперечливих відношень відповідно до Вашого варіанту.

Теоретичні основи:

Упорядкована пара предметів – це сукупність, що складається із двох предметів, розташованих у деякому певному порядку. При цьому впорядкована пара має наступні властивості:

а) для будь-яких двох предметів x і y існує об'єкт, який можна позначити як $\langle x, y \rangle$ названий упорядкованою парою;

б) якщо $\langle x, y \rangle$ і $\langle u, v \rangle$ – упорядковані пари, то $\langle x, y \rangle = \langle u, v \rangle$ тоді і тільки тоді, коли x = u, y = v .

Бінарним (або *двомісним*) відношенням R називають підмножину впорядкованих пар, тобто множину, кожен елемент якої є впорядкованою парою. Якщо R є деяким відношенням, це записують як $\langle x, y \rangle \in R$ або xRy.

Один з типів відношень − це множина всіх таких пар x, y, що x є елементом деякої фіксованої множини X , а y − елементом деякої фіксованої множини Y . Таке відношення називають прямим або *декартовим добутком*.

Операції над відношеннями

Оскільки відношення на множині A задають підмножинами $R \subseteq A \times B$, то для них визначні ті ж операції, що й над множинами, а саме:

1. Об'єднання: .

2. Перетин: { }2121 ∈∈=∩ Rb,aiRb,ab,aRR .

3. Різниця: □ □1 2 1 2R \ R a,b a,b R i a,b R□ □ □ .

4. Доповнення: RUR \□ , де BAU □□ .

Крім того, необхідно визначити інші операції над бінарними відношеннями.

5. Обернене відношення R^{-1}

Якщо $\langle a,b\rangle \in R$, – відношення, то відношення R^{-1} називають **оберненим відношенням** до даного відношення R тоді й тільки тоді, коли $R^{-1} = \{\langle b,a\rangle | \langle a,b\rangle \in R\}$.

Загальний порядок виконання лабораторної роботи

1. Вибрати номер Z індивідуального варіанта відповідно до виразу:

Z=(i+G%60)%30+1, де i – номер у списку групи, G – числова складова назви групи.

2. За індивідуальним варіантом побудувати алгоритм, який встановлює одночасно несуперечливі відношення S та R для двох довільних множин.

3. Відношення S та R представити у вигляді матриць або графів.

4. Результат операцій над відношеннями також представити у вигляді матриць або графів.

Вимоги до інтерфейсу

Програма повинна складатися з 4-х вікон.

Вміст вікна №1

1.Головне меню, яке повинно включати виклик вікна №2, вікна №3 та вікна №4.

2. Віджети виводу П.І.Б студента, номера групи, номера у списку та віджет виводу результатів обчислення варіанту відповідно да програми, що задана у завданні – (пункт 1 загального порядку виконання лабораторної роботи).

Вміст вікна №2

3. Віджет Listbox зі списком жіночих імен з можливістю вибору з нього довільної кількості імен з наступним копіюванням цих імен у віджет Listbox списку множини A або B, в залежності від стану віджета Radiobutton, який вказує у який список відбувається копіювання імені.

4. Віджет Listbox зі списком чоловічих імен з можливістю вибору з нього довільної кількості імен з наступним копіюванням цих імен у віджет Listbox списку множини A або B, в залежності від стану віджета Radiobutton, який вказує у який список відбувається копіювання імені.

5. Віджети Button для збереження у файл, зчитування з файлу та очищення множини A.

6. Віджети Button для збереження у файл, зчитування з файлу та очищення множини B.

Вміст вікна №3

1. Віджети Listbox для відображення елементів множин A і B .

2. Матричне або графічне представлення відношення S, яке сформоване вашим алгоритмом

3. Матричне або графічне представлення відношення R, яке сформоване вашим алгоритмом з урахуванням несуперечливості з відношенням S.

Вміст вікна №4

1. Матричне або графічне представлення відношення $R \cup S$

2. Матричне або графічне представлення відношення $R \cap S$

3. Матричне або графічне представлення відношення $R \setminus S$

4. Матричне або графічне представлення відношення $U \setminus R$

5. Матричне або графічне представлення відношення S^{-1}

Виконання індивідуальних завдань

Программа обчислення варіанта:

```
My Group - 45
My number - 23
Var 9
```

9	aSb, якщо a мати b.	aRb, якщо a сестра b.

модуль aSb_aRb

```
import random
import networkx as nx

def a_mother_b(a_set, b_set, female_names):
    s = set()
    count_of_child = []
    for i in range(int(len(a_set) * 2)):
        mother = random.choice(list(a_set))
        child = random.choice(list(b_set))
        if mother not in female_names or mother == child:
            continue
        if child not in count_of_child:
            s.add((mother, child))
            count_of_child.append(child)
    return s

def a_sister_b(a_set, b_set, relation_ss, female_names):
    r = set()
    mother_children = {}
    for mother, child in relation_ss:
        mother_children[mother] = mother_children.get(mother, []) + [child]
    for i in range(len(a_set) * 5):
        sister1 = random.choice(list(b_set))
        sister2 = random.choice(list(b_set))
        if (sister1 not in female_names or sister2 not in female_names or
                sister1 == sister2):
            continue
        if (sister1, sister2) in r or (sister2, sister1) in r:
            continue
        is_sisters = False
        for mother, children in mother_children.items():
            if sister1 in children and sister2 in children:
                is_sisters = True
                break
```

```
        if is_sisters:
            r.add((sister1, sister2))
    return r

def build_matrix(a_elements, b_elements, relation):
    matrix = [[0] * len(b_elements) for _ in range(len(a_elements))]
    a_list = list(a_elements)
    b_list = list(b_elements)
    for i, elem1 in enumerate(a_list):
        for j, elem2 in enumerate(b_list):
            if (elem1, elem2) in relation:
                matrix[i][j] = 1
    return matrix

def print_matrix(matrix, a_elements, b_elements):
    a_list = list(a_elements)
    b_list = list(b_elements)
    result = "  " + " ".join([e[:3] for e in b_list]) + "\n"
    for i, row in enumerate(matrix):
        result += f"{a_list[i][:3]} {row}\n"
    return result

def create_graph(matrix, a_elements, b_elements):
    G = nx.DiGraph()
    a_list = list(a_elements)
    b_list = list(b_elements)
    G.add_nodes_from(a_list)
    G.add_nodes_from(b_list)
    for i, row in enumerate(matrix):
        for j, val in enumerate(row):
            if val == 1:
                G.add_edge(a_list[i], b_list[j])
    return G

def get_universe(a_elements, b_elements):
    return {(a, b) for a in a_elements for b in b_elements}
```

Головний файл програми

```
import tkinter as tk
from aSb_aRb import a_mother_b, a_sister_b, build_matrix, create_graph,
get_universe, print_matrix
import networkx as nx
from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg
import matplotlib.pyplot as plt
```

Перше вікно

```
class App:
    def __init__(self, app_root):
        self.root = app_root
        self.root.title("Вікно №1")
        self.root.geometry("400x400")

        self.fame_name = ["Тоня", "Анна", "Марія", "Оля", "Каріна", "Поліна",
"Христина", "Олена"]
        self.a = None
        self.b = None

        self.group_number = 45
        self.number_list = 23
        self.window_2, self.window_3, self.window_4, self.window_5 = None, None,
None, None

        menu_bar = tk.Menu(self.root)
        window_menu = tk.Menu(menu_bar, tearoff=0)
        menu_bar.add_cascade(label="Вікна", menu=window_menu, font=("Arial",
11))
        window_menu.add_command(label="Вікно № 2",
command=self.open_window2)
        window_menu.add_command(label="Вікно № 3",
command=self.open_window3)
        window_menu.add_command(label="Вікно № 4",
command=self.open_window4)
        self.root.config(menu=menu_bar)

        self.root.columnconfigure(0, weight=1)
        self.root.columnconfigure(1, weight=1)
```

```
        self.info_frame = tk.Frame(self.root, bg="white", bd=1, relief="solid")
        self.info_frame.grid(column=0, pady=20, padx=20, columnspan=2)

        self.name_label = tk.Label(self.info_frame, text="П.І.Б. студента: Унятицький
Антон Дмитрович", bg="white")
        self.name_label.grid(row=1, column=1, padx=5, pady=10)

        self.group_label = tk.Label(self.info_frame, text=f"Номер групи:
{self.group_number}", bg="white")
        self.group_label.grid(row=2, column=1, padx=5)

        self.list_number_label = tk.Label(self.info_frame, text=f"Номер у списку:
{self.number_list}", bg="white")
        self.list_number_label.grid(row=3, column=1, padx=5)

        self.variant_result_label = tk.Label(self.info_frame,
                                             text=f"Результат обчислення варіанту:
{self.generate_variant()}",
                                             bg="white")
        self.variant_result_label.grid(row=4, column=1, padx=5, pady=5)

    def open_window2(self):
        self.window_2 = tk.Toplevel(self.root)
        Window2(self.window_2)

    def open_window3(self):
        self.a = []
        try:
            with open('mnog_a.txt', 'r', encoding="utf-8") as file:
                self.a = [line.strip() for line in file if line.strip()]
        except FileNotFoundError:
            self.a = ["Анна", "Марія", "Олена"]

        self.b = []
        try:
            with open('mnog_b.txt', 'r', encoding="utf-8") as file:
                self.b = [line.strip() for line in file if line.strip()]
        except FileNotFoundError:
            self.b = ["Оля", "Катя", "Петро"]
```

```
        self.window_3 = tk.Toplevel(self.root)
        Window3(self.window_3, self.a, self.b, self.fame_name)

    def open_window4(self):
        self.a = []
        try:
            with open('mnog_a.txt', 'r', encoding="utf-8") as file:
                self.a = [line.strip() for line in file if line.strip()]
        except FileNotFoundError:
            self.a = ["Анна", "Марія", "Олена"]

        self.b = []
        try:
            with open('mnog_b.txt', 'r', encoding="utf-8") as file:
                self.b = [line.strip() for line in file if line.strip()]
        except FileNotFoundError:
            self.b = ["Оля", "Катя", "Петро"]

        self.window_4 = tk.Toplevel(self.root)
        Window4(self.window_4, self.a, self.b, self.fame_name)

    def generate_variant(self):
        result = (self.number_list + self.group_number % 60) % 30 + 1
        return result

# Друге вікно
class Window2:
    def __init__(self, window):
        self.root = window
        self.root.title("Вікно 2")
        self.root.geometry("600x600")

        self.woman = ["Тоня", "Анна", "Марія", "Оля", "Каріна", "Поліна",
"Христина", "Олена"]
        self.man = ["Дмитро", "Павло", "Кирило", "Мефодій", "Антон", "Богдан",
"Артем"]

        self.listbox = tk.Listbox(self.root)
        for item in self.woman:
```

```
            self.listbox.insert(tk.END, item)
        self.listbox.grid(column=2, pady=20, padx=20)
        self.listbox.bind("<<ListboxSelect>>", self.add_selected_item)

        self.listbox_man = tk.Listbox(self.root)
        for item_m in self.man:
            self.listbox_man.insert(tk.END, item_m)
        self.listbox_man.grid(column=3, row=0, pady=20, padx=20)
        self.listbox_man.bind("<<ListboxSelect>>", self.add_selected_item)

        self.listbox_a = tk.Listbox(self.root)
        self.listbox_a.grid(column=2, row=1, padx=20)

        self.listbox_b = tk.Listbox(self.root)
        self.listbox_b.grid(column=2, row=2, pady=20, padx=20)

        self.clear_frame_a = tk.Frame(self.root)
        self.clear_frame_a.grid(column=3, row=1, pady=20)
        self.clear_frame_b = tk.Frame(self.root)
        self.clear_frame_b.grid(column=3, row=2, pady=20)

        self.selected_option = tk.StringVar(value="Множина A")
        self.radio_btn_a = tk.Radiobutton(self.clear_frame_a, text='Множина A',
                                          variable=self.selected_option, value='Множина A')
        self.radio_btn_a.grid(row=0, column=0, pady=5)

        self.radio_btn_b = tk.Radiobutton(self.clear_frame_b, text='Множина B',
                                          variable=self.selected_option, value='Множина B')
        self.radio_btn_b.grid(row=0, column=0, pady=5)

        self.btn_save_a = tk.Button(self.clear_frame_a, text="Зберегти множину A",
command=self.save_a)
        self.btn_save_a.grid(row=1, column=0, pady=5)
        self.btn_save_b = tk.Button(self.clear_frame_b, text="Зберегти множину B",
command=self.save_b)
        self.btn_save_b.grid(row=1, column=0, pady=5)

        self.clear_a_btn = tk.Button(self.clear_frame_a, text="Очистити множину A",
                                     command=lambda: self.listbox_a.delete(0, tk.END))
        self.clear_a_btn.grid(row=2, column=0, pady=5)
```

```
        self.clear_b_btn = tk.Button(self.clear_frame_b, text="Очистити множину B",
                                     command=lambda: self.listbox_b.delete(0, tk.END))
        self.clear_b_btn.grid(row=2, column=0, pady=5)

        self.read_a_btn = tk.Button(self.clear_frame_a, text="Зчитати A",
command=self.read_a)
        self.read_a_btn.grid(row=3, column=0, pady=5)
        self.read_b_btn = tk.Button(self.clear_frame_b, text="Зчитати B",
command=self.read_b)
        self.read_b_btn.grid(row=3, column=0, pady=5)

    def read_a(self):
        with open('mnog_a.txt', 'r', encoding="utf-8") as file:
            for line in file:
                self.listbox_a.insert(tk.END, line.strip())

    def read_b(self):
        with open('mnog_b.txt', 'r', encoding="utf-8") as file:
            for line in file:
                self.listbox_b.insert(tk.END, line.strip())

    def save_a(self):
        items = self.listbox_a.get(0, tk.END)
        with open('mnog_a.txt', 'w', encoding='utf-8') as file:
            for item in items:
                file.write(f"{item}\n")
        tk.Label(self.clear_frame_a, text="Збережено").grid(row=1, column=1,
pady=5)

    def save_b(self):
        items = self.listbox_b.get(0, tk.END)
        with open('mnog_b.txt', 'w', encoding='utf-8') as file:
            for item in items:
                file.write(f"{item}\n")
        tk.Label(self.clear_frame_b, text="Збережено").grid(row=1, column=1,
pady=5)

    def add_selected_item(self, event):
        widget = event.widget
        if widget == self.listbox:
```

```
            selected_indices = widget.curselection()
            item = widget.get(selected_indices[0]) if selected_indices else None
            item_man = None
        elif widget == self.listbox_man:
            selected_indices_2 = widget.curselection()
            item = None
            item_man = widget.get(selected_indices_2[0]) if selected_indices_2 else
None
        else:
            return

        if self.selected_option.get() == 'Множина A':
            if item:
                self.listbox_a.insert(tk.END, item)
            if item_man:
                self.listbox_a.insert(tk.END, item_man)
        elif self.selected_option.get() == 'Множина B':
            if item:
                self.listbox_b.insert(tk.END, item)
            if item_man:
                self.listbox_b.insert(tk.END, item_man)

# Третє вікно
class Window3:
    def __init__(self, window, a, b, fem_name):
        self.root = window
        self.root.title("Вікно 3")
        self.root.geometry("1000x800")

        self.list_a = a
        self.list_b = b
        self.fem_name = fem_name

        self.display_listbox("A:", self.list_a, 1)
        self.display_listbox("B:", self.list_b, 2)

        s_relation = a_mother_b(self.list_a, self.list_b, self.fem_name)
        r_relation = a_sister_b(self.list_b, self.list_b, s_relation, self.fem_name)

        s_matrix = build_matrix(self.list_a, self.list_b, s_relation)
```

```
        r_matrix = build_matrix(self.list_b, self.list_b, r_relation)

        tk.Label(self.root, text="Матриця S (Мати-Дитина):").grid(row=2, column=1,
pady=5)
        tk.Label(self.root, text=print_matrix(s_matrix, self.list_a,
self.list_b)).grid(row=3, column=1, pady=5)

        tk.Label(self.root, text="Матриця R (Сестра-Сестра):").grid(row=2,
column=2, pady=5)
        tk.Label(self.root, text=print_matrix(r_matrix, self.list_b,
self.list_b)).grid(row=3, column=2, pady=5)

        self.display_graph(s_matrix, self.list_a, self.list_b, "Граф S (Мати-Дитина)",
row=4, column=1)
        self.display_graph(r_matrix, self.list_b, self.list_b, "Граф R (Сестра-Сестра)",
row=4, column=2)

    def display_listbox(self, label, items, column):
        tk.Label(self.root, text=label).grid(row=0, column=column, pady=5)
        listbox = tk.Listbox(self.root)
        listbox.grid(row=1, column=column, padx=5, pady=5)
        for item in items:
            listbox.insert(tk.END, item)

    def display_graph(self, matrix, a_elements, b_elements, title, row, column):
        G = create_graph(matrix, a_elements, b_elements)
        fig = plt.Figure(figsize=(4, 3))
        ax = fig.add_subplot(111)
        pos = nx.spring_layout(G)
        nx.draw(G, pos, ax=ax, with_labels=True, node_color='lightblue',
                node_size=500, font_size=8, arrows=True)
        ax.set_title(title)
        canvas = FigureCanvasTkAgg(fig, master=self.root)
        canvas.draw()
        canvas.get_tk_widget().grid(row=row, column=column, padx=10, pady=10)
```

Четверте вікно

```
class Window4:
    def __init__(self, window, a, b, fem_name):
        self.root = window
```

```
        self.root.title("Вікно 4")
        self.root.geometry("1200x1000")

        self.list_a = a
        self.list_b = b
        self.fem_name = fem_name

        s_relation = a_mother_b(self.list_a, self.list_b, self.fem_name)
        r_relation = a_sister_b(self.list_b, self.list_b, s_relation, self.fem_name)

        u_relation = get_universe(self.list_a, self.list_b)

        rus_relation = r_relation.union(s_relation)
        rus_matrix = build_matrix(self.list_a + self.list_b, self.list_b, rus_relation)
        self.display_graph(rus_matrix, self.list_a + self.list_b, self.list_b, "R ∪ S",
row=0, column=0)

        rns_relation = r_relation.intersection(s_relation)
        rns_matrix = build_matrix(self.list_a + self.list_b, self.list_b, rns_relation)
        self.display_graph(rns_matrix, self.list_a + self.list_b, self.list_b, "R ∩ S",
row=0, column=1)

        r_minus_s_relation = r_relation.difference(s_relation)
        r_minus_s_matrix = build_matrix(self.list_b, self.list_b, r_minus_s_relation)
        self.display_graph(r_minus_s_matrix, self.list_b, self.list_b, "R \\ S", row=0,
column=2)

        u_minus_r_relation = u_relation.difference(r_relation)
        u_minus_r_matrix = build_matrix(self.list_a + self.list_b, self.list_b,
u_minus_r_relation)
        self.display_graph(u_minus_r_matrix, self.list_a + self.list_b, self.list_b, "U \\
R", row=1, column=0)

        s_inverse_relation = {(b, a) for (a, b) in s_relation}
        s_inverse_matrix = build_matrix(self.list_b, self.list_a, s_inverse_relation)
        self.display_graph(s_inverse_matrix, self.list_b, self.list_a, "S⁻¹", row=1,
column=1)

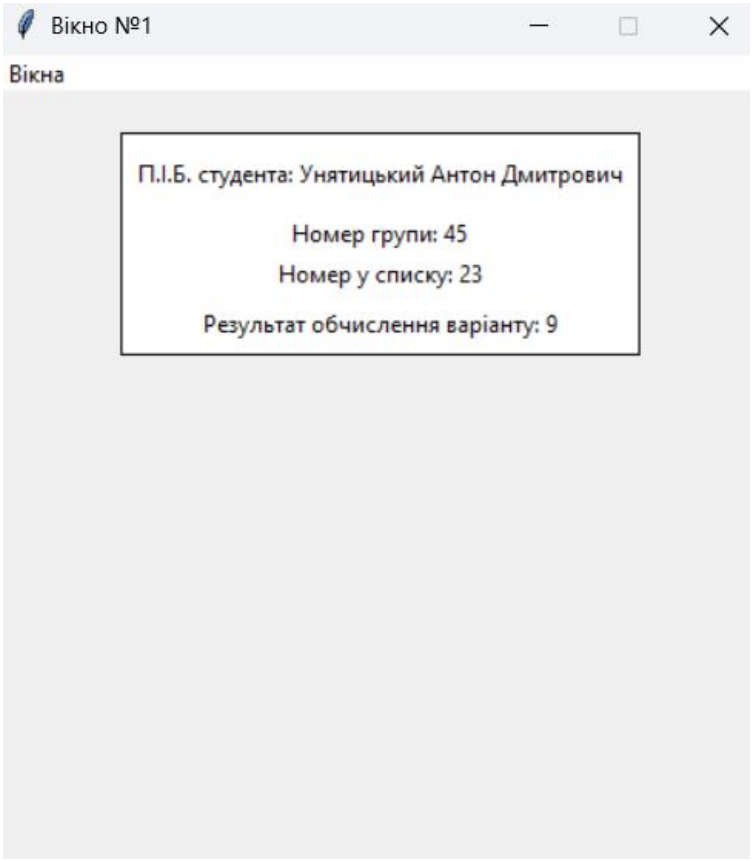
    def display_graph(self, matrix, a_elements, b_elements, title, row, column):
        G = create_graph(matrix, a_elements, b_elements)
```

```
        fig = plt.Figure(figsize=(4, 3))
        ax = fig.add_subplot(111)
        pos = nx.spring_layout(G)
        nx.draw(G, pos, ax=ax, with_labels=True, node_color='lightblue',
                node_size=500, font_size=8, arrows=True)
        ax.set_title(title)
        canvas = FigureCanvasTkAgg(fig, master=self.root)
        canvas.draw()
        canvas.get_tk_widget().grid(row=row, column=column, padx=10, pady=10)

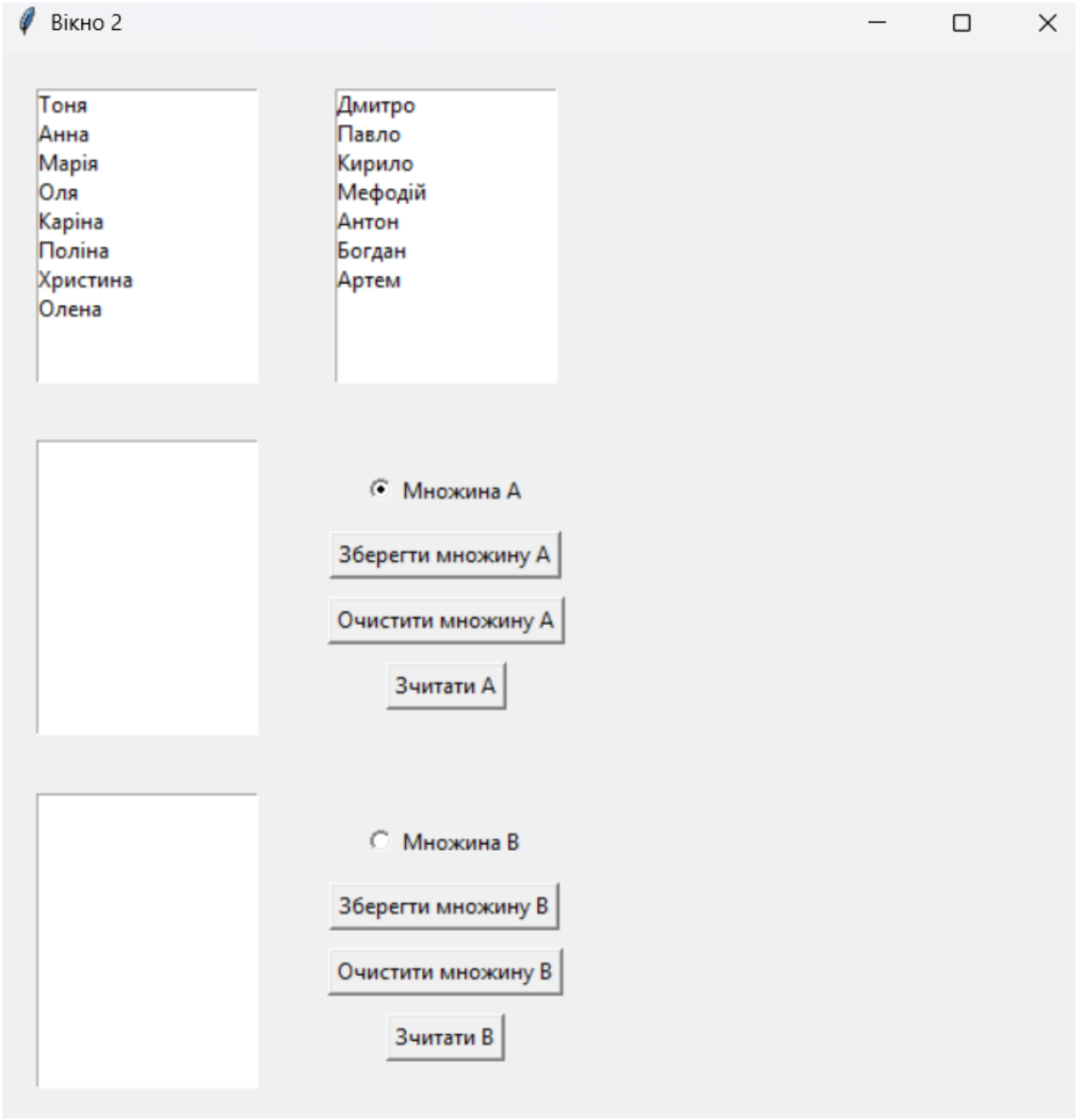

if __name__ == "__main__":
    root = tk.Tk()
    root.resizable(False, False)
    root.protocol("WM_DELETE_WINDOW", root.quit)
    app = App(root)
    root.mainloop()
```

Скриншоти виконання програми :

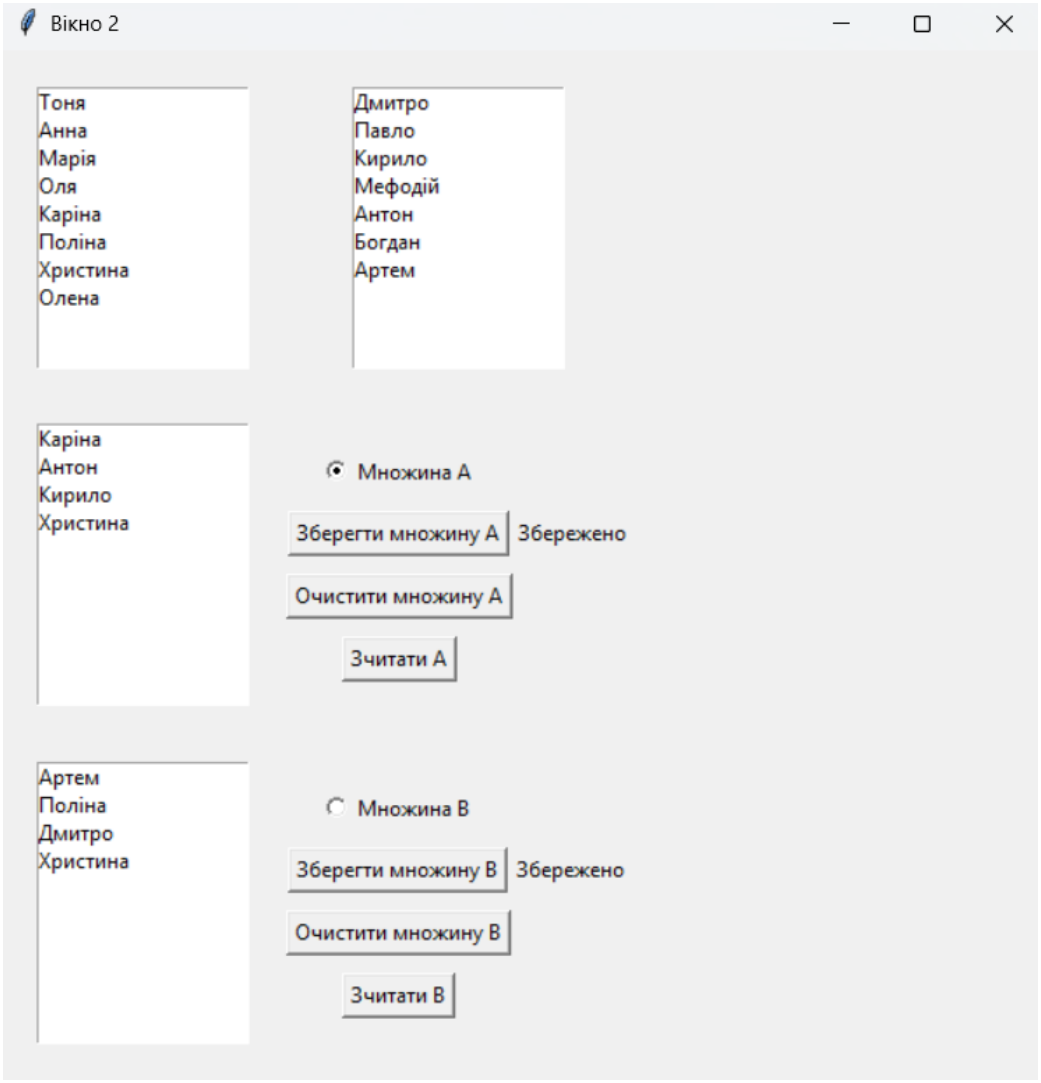
Вікно 1

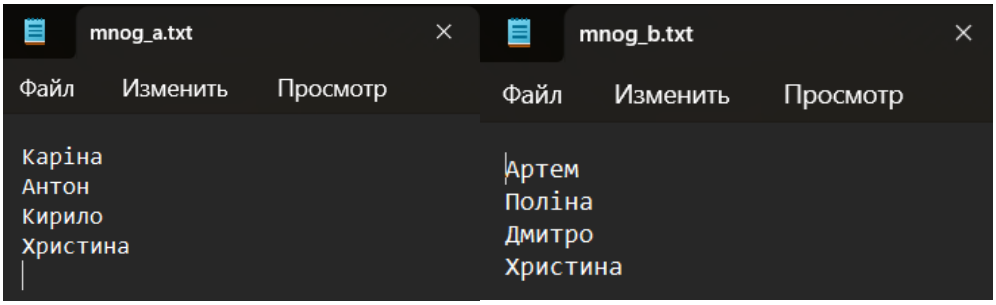


Вікно 2

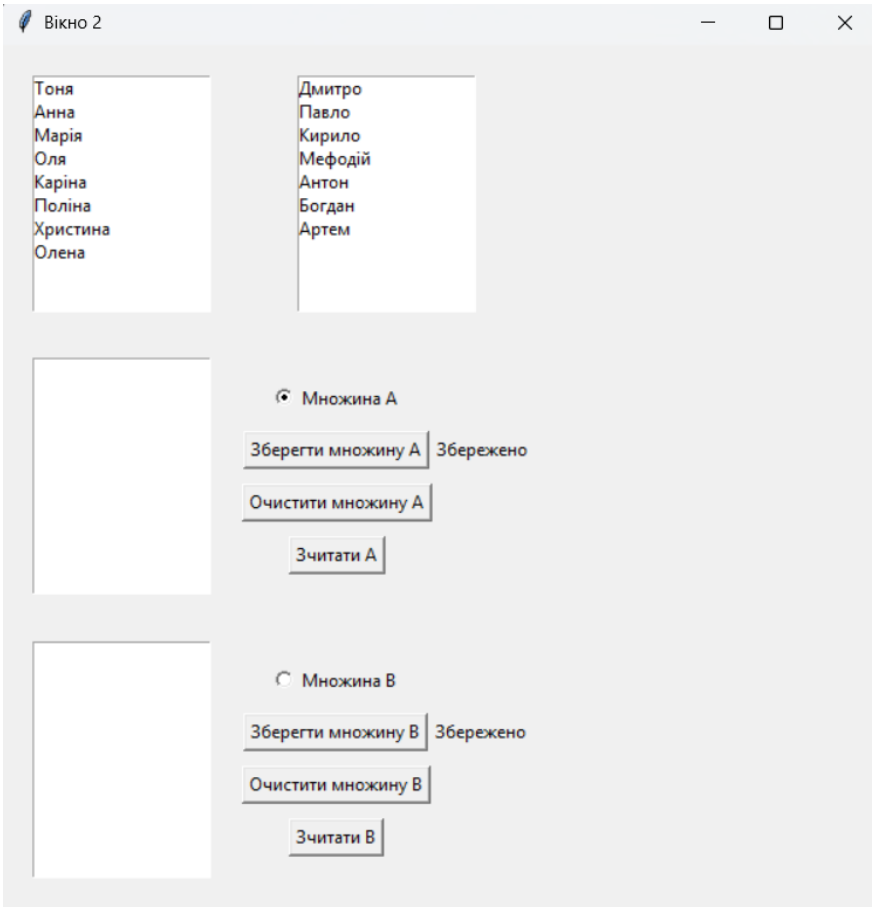


Додавання імен до множин та збереження множин

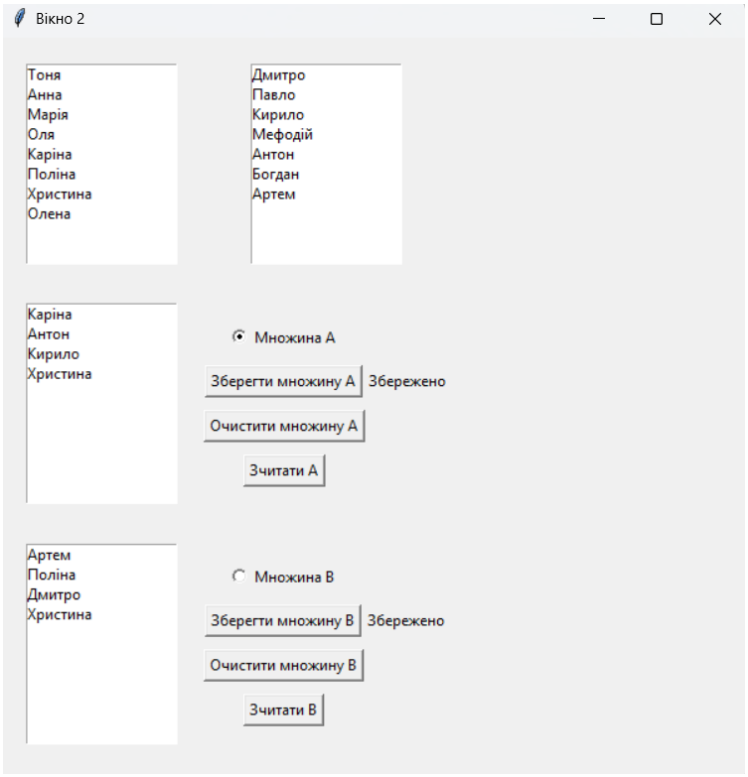




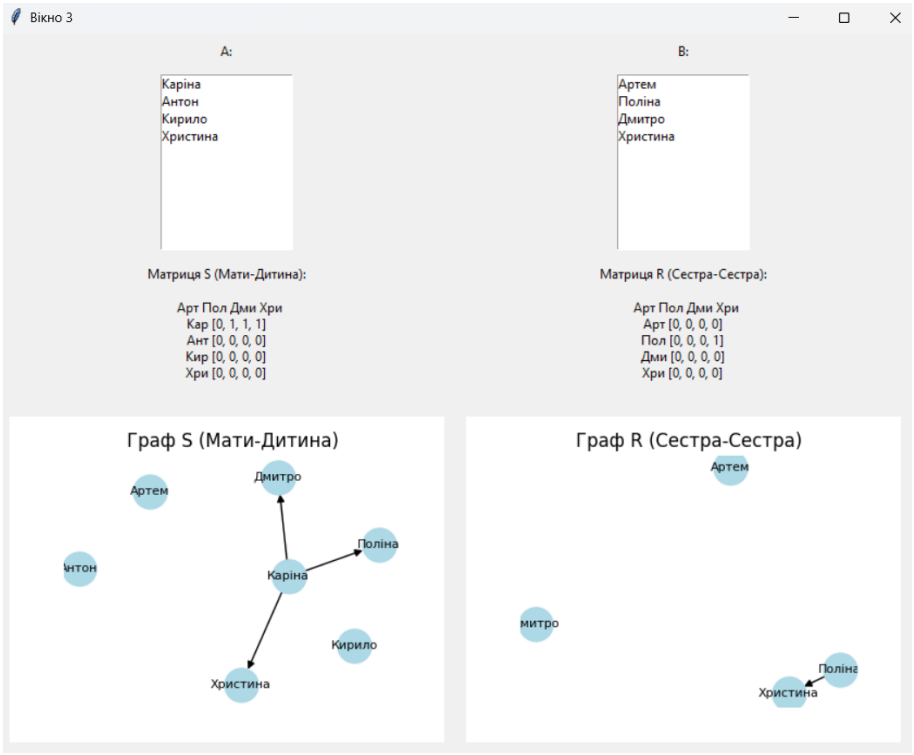
Очищення множин



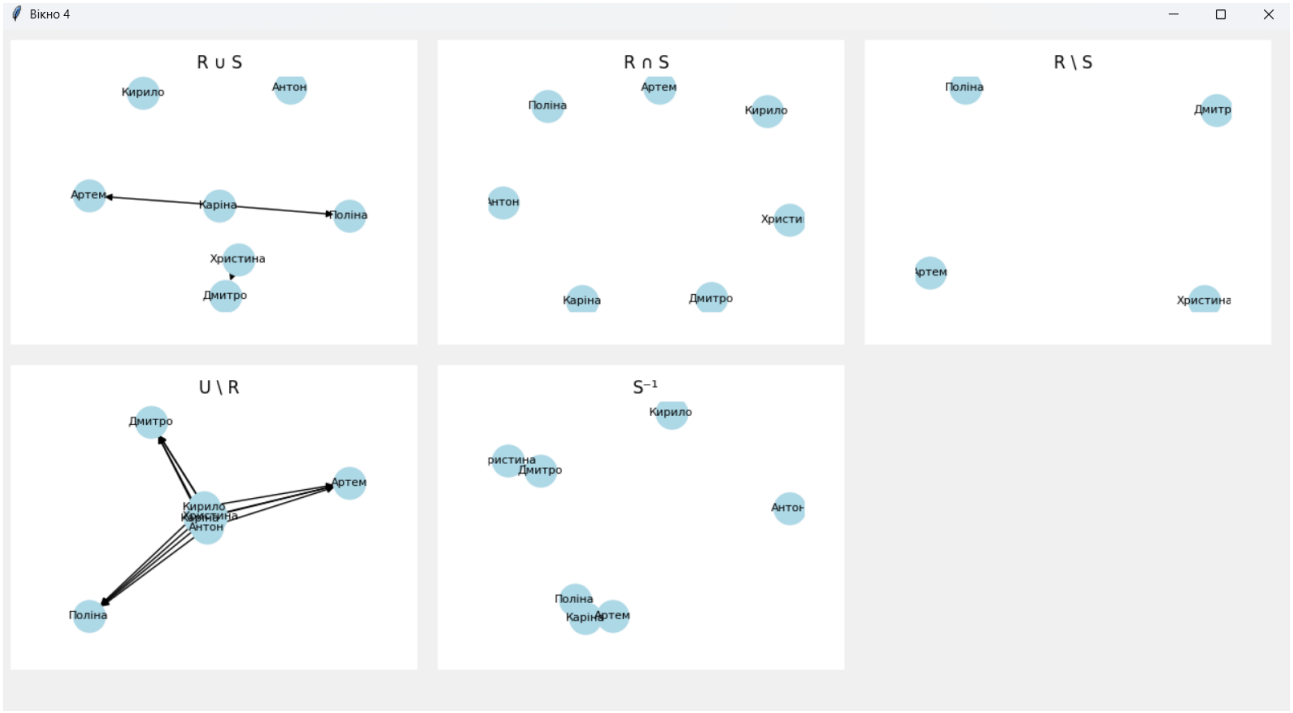
Зчитування множин з файлів



Вікно 3



Вікно 4



Висновок. На цій лабораторній роботі я вивчив бінарні відношення, створив програму , яка формує відношення у відповідності з моїм варіантом завдання та виконує операції над цими відношеннями. Також ознайомився з основними властивостями над бінарними відношеннями.